Zagazig University

Faculty of Engineering

Computer and Systems Engineering Department

CSE 323: Operating Systems

Introduction to Linux Kernel Modules

Programming Project #1

Name	**Code**
Youssef Reda Elsayed Mouawad	20812023100544

Academic Year

2026

Contents

Introduction to Linux Kernel Modules **1**
- Project Overview . . . 1
- Module Initialization and Cleanup . . . 1
- Compilation Using Make . . . 1
- Loading, Inspecting, and Removing Modules . . . 2
- The /proc Filesystem . . . 2
- Jiffies and the HZ Constant . . . 2

Module 1: proc-hello **3**
- Objective . . . 3
- Implementation . . . 3
- Compilation and Execution . . . 3
- Testing and Results . . . 4

Module 2: proc-jiffies **5**
- Objective . . . 5
- Implementation . . . 5
- Compilation and Execution . . . 5
- Testing and Results . . . 6

Module 3: proc-seconds **7**
- Objective . . . 7
- Implementation . . . 7
- Compilation and Execution . . . 7
- Testing and Results . . . 8

Conclusion **9**

References **10**

Introduction to Linux Kernel Modules

Project Overview

Linux kernel modules are pieces of code that can be loaded into and removed from a running Linux kernel without rebuilding or restarting the entire operating system. They provide a practical way to extend kernel functionality and interact with kernel facilities.

The objective of this project is to gain practical experience with Linux kernel module development, compilation, loading, removal, and communication with user space through the `/proc` virtual filesystem.

The project implements three modules:

- **proc-hello:** Creates a `/proc/hello` entry that returns a greeting message.
- **proc-jiffies:** Creates a `/proc/jiffies` entry that reports the current value of the kernel's `jiffies` counter.
- **proc-seconds:** Creates a `/proc/seconds` entry that reports the elapsed time in seconds since the module was loaded.

Module Initialization and Cleanup

A kernel module typically defines two functions: an initialization function and a cleanup function.

- **Initialization:** Runs when the module is loaded. It performs the required setup and returns zero on success.
- **Cleanup:** Runs when the module is removed. It releases resources allocated by the module, such as entries created in the `/proc` filesystem.

The `module_init()` and `module_exit()` macros register these functions with the kernel. Kernel messages can be written using `pr_info()` or `printk()` and inspected with the `dmesg` command.

Compilation Using Make

External kernel modules are commonly compiled using the kernel build system, known as Kbuild. Each module directory contains a Makefile that specifies the module's object file and invokes the build system.

The following commands illustrate the build process:

```
make
make clean
```

The `make` command compiles the module against the build files for the running kernel. The `make clean` command removes generated build artifacts.

Loading, Inspecting, and Removing Modules

The following commands are used to manage a compiled module. Run them from the directory containing the corresponding `.ko` file.

```
sudo insmod proc_hello.ko
lsmod
dmesg | tail
sudo rmmod proc_hello
```

- `insmod` inserts a compiled module into the kernel.
- `lsmod` lists currently loaded kernel modules.
- `dmesg` displays messages from the kernel log buffer.
- `rmmod` removes a loaded module.

The module name passed to `rmmod` is the module's name, not the filename with the `.ko` extension. Administrative privileges are generally required to load and remove modules.

The /proc Filesystem

The `/proc` filesystem is a virtual filesystem that provides an interface for accessing selected kernel and process information. Its entries are generated dynamically and do not behave like ordinary files stored on disk.

A kernel module can create a `/proc` entry and provide a read callback that is invoked when a user-space program reads it. For example:

```
cat /proc/hello
```

In the implemented modules, the callback prepares a message in kernel memory and uses `copy_to_user()` to transfer the data to a user-space buffer. The module's cleanup function removes the corresponding `/proc` entry.

Jiffies and the HZ Constant

The kernel maintains a counter named `jiffies`, which represents timer ticks since system boot. The `HZ` constant defines the configured number of timer ticks per second.

The elapsed time in seconds can therefore be calculated using:

```
elapsed_seconds = (jiffies - start_jiffies) / HZ;
```

Here, `start_jiffies` stores the counter value captured when the module is loaded. Subtracting it from the current counter gives the number of ticks elapsed during the module's lifetime. Dividing by `HZ` converts the result to seconds.

Module 1: proc-hello

Objective

The purpose of this module is to create a `/proc/hello` entry that returns a fixed greeting message whenever it is read. This module demonstrates the basic interaction between a kernel module and a user-space application through the `/proc` filesystem.

Implementation

During initialization, the module creates the `/proc/hello` entry and registers a read callback using `proc_create()` and `struct proc_ops`. If creation fails, initialization returns an error.

When the entry is read, the callback prepares the greeting message and copies it to the user-space buffer using `copy_to_user()`. The file position is updated to indicate that the message has been consumed. During cleanup, the entry is removed.

Compilation and Execution

Run the following commands from the `modules/proc-hello` directory:

```
make
sudo insmod proc_hello.ko
cat /proc/hello
sudo rmmod proc_hello
```

The expected output of the read operation is:

```
Hello from the Linux Kernel via /proc!
```

The module can also be inspected through the kernel log:

```
dmesg | tail
```

Testing and Results

The module was built and loaded into the running kernel. Reading `/proc/hello` returned the expected greeting message. After removing the module, the corresponding `/proc` entry was removed as part of the cleanup process.

```
youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello$ make
make -C /lib/modules/7.1.5+kali-amd64/build M=/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello modules
make[1]: Entering directory '/usr/src/linux-headers-7.1.5+kali-amd64'
make[2]: Entering directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello'
warning: pahole version differs from the one used to build the kernel
  The kernel was built with: 131
  You are using:             132
  CC [M]  proc_hello.o
  MODPOST Module.symvers
  CC [M]  proc_hello.mod.o
  CC [M]  .module-common.o
  LD [M]  proc_hello.ko
  BTF [M] proc_hello.ko
make[2]: Leaving directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello'
make[1]: Leaving directory '/usr/src/linux-headers-7.1.5+kali-amd64'

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello$ sudo insmod proc_hello.ko

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello$ dmesg | tail -n 1
[20449.269263] /proc/hello created

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello$ cat /proc/hello
Hello from the Linux Kernel via /proc!

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello$ sudo rmmod proc_hello

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello$ dmesg | tail -n 1
[20469.274124] /proc/hello removed

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello$ make clean
make -C /lib/modules/7.1.5+kali-amd64/build M=/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello clean
make[1]: Entering directory '/usr/src/linux-headers-7.1.5+kali-amd64'
make[2]: Entering directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello'
  CLEAN   Module.symvers
make[2]: Leaving directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-hello'
make[1]: Leaving directory '/usr/src/linux-headers-7.1.5+kali-amd64'
```

Figure 1: Testing the proc-hello module and reading its /proc entry.

Module 2: proc-jiffies

Objective

The purpose of this module is to create a `/proc/jiffies` entry that reports the current value of the kernel's `jiffies` counter when the entry is read.

Implementation

The module uses the kernel's jiffies-related definitions to access the current counter value. Its initialization function creates the `/proc/jiffies` entry and registers the read callback.

When a user-space application reads the entry, the callback formats the current counter value as text and transfers it to user space. The cleanup function removes the entry when the module is unloaded.

The value of `jiffies` increases as timer ticks occur. Consequently, reading the entry at different times may return different values.

Compilation and Execution

Run the following commands from the `modules/proc-jiffies` directory:

```
make
sudo insmod proc_jiffies.ko
cat /proc/jiffies
cat /proc/jiffies
sudo rmmod proc_jiffies
```

The output is a numeric value representing the current `jiffies` counter. The exact value depends on the system's uptime and timer activity, so it should not be hard-coded in the report.

Testing and Results

The module was compiled and loaded successfully. Reading `/proc/jiffies` returned the current counter value, allowing the kernel's timer-tick counter to be observed from user space. The entry was designed to be removed when the module is unloaded.

```
youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies$ make
make -C /lib/modules/7.1.5+kali-amd64/build M=/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies modules
make[1]: Entering directory '/usr/src/linux-headers-7.1.5+kali-amd64'
make[2]: Entering directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies'
warning: pahole version differs from the one used to build the kernel
  The kernel was built with: 131
  You are using:             132
  CC [M]  proc_jiffies.o
  MODPOST Module.symvers
  CC [M]  proc_jiffies.mod.o
  CC [M]  .module-common.o
  LD [M]  proc_jiffies.ko
  BTF [M] proc_jiffies.ko
make[2]: Leaving directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies'
make[1]: Leaving directory '/usr/src/linux-headers-7.1.5+kali-amd64'

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies$ sudo insmod proc_jiffies.ko

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies$ dmesg | tail -n 1
[20524.470149] /proc/jiffies created

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies$ cat /proc/jiffies
4300025199

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies$ sudo rmmod proc_jiffies

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies$ dmesg | tail -n 1
[20540.722909] /proc/jiffies removed

youssef@Hacker:~/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies$ make clean
make -C /lib/modules/7.1.5+kali-amd64/build M=/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies clean
make[1]: Entering directory '/usr/src/linux-headers-7.1.5+kali-amd64'
make[2]: Entering directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies'
  CLEAN   Module.symvers
make[2]: Leaving directory '/home/youssef/Desktop/Github_Repos/linux-kernel-modules/modules/proc-jiffies'
make[1]: Leaving directory '/usr/src/linux-headers-7.1.5+kali-amd64'
```

Figure 2: Testing the proc-jiffies module and displaying the current counter.

Module 3: proc-seconds

Objective

The purpose of this module is to create a `/proc/seconds` entry that reports the number of seconds elapsed since the module was loaded.

Implementation

The module stores the value of `jiffies` at initialization in a variable named `start_jiffies`. Whenever the `/proc/seconds` entry is read, the callback calculates the difference between the current counter and the stored starting value.

The elapsed time is calculated using:

```
elapsed_seconds = (jiffies - start_jiffies) / HZ;
```

The subtraction gives the number of timer ticks since the module was loaded, and division by `HZ` converts that difference to seconds. The result is formatted as text and returned to user space. The module removes the `/proc/seconds` entry during cleanup.

Compilation and Execution

Run the following commands from the `modules/proc-seconds` directory:

```
make
sudo insmod proc_seconds.ko
cat /proc/seconds
sleep 2
cat /proc/seconds
sudo rmmod proc_seconds
```

The reported value should generally increase as time passes. Because the calculation uses integer division, the value changes in whole seconds rather than continuously.

Testing and Results

The module was compiled and loaded successfully. Reading `/proc/seconds` reported the elapsed time since loading. Reading it again after a delay demonstrated the change in elapsed time. The module's cleanup function removes its `/proc` entry when the module is unloaded.

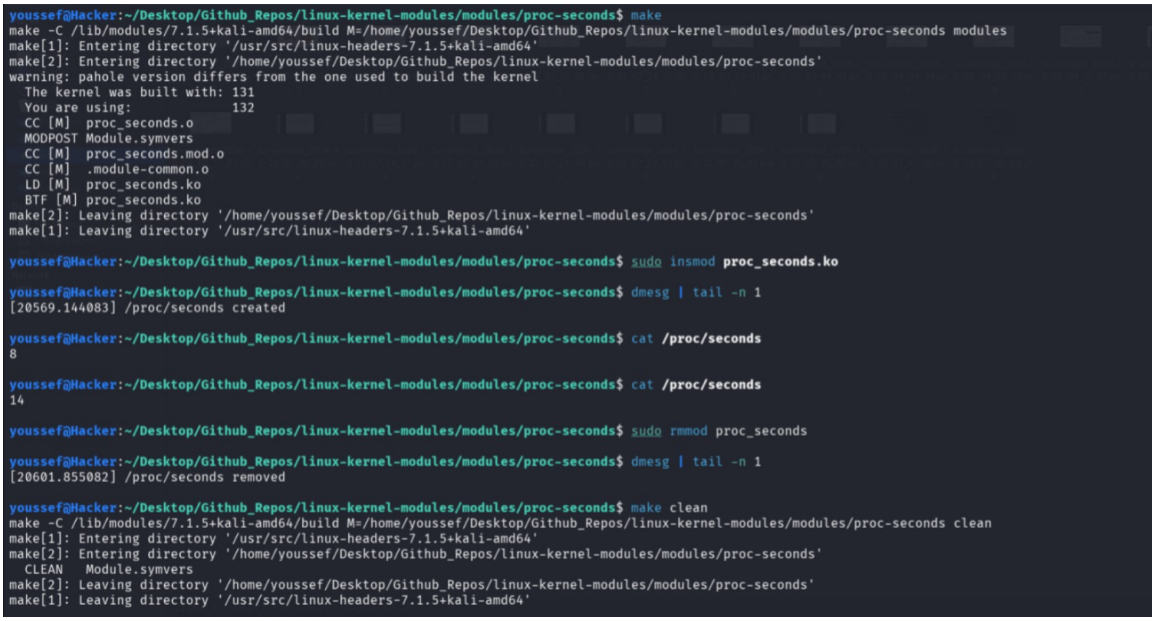


Figure 3: Testing the proc-seconds module and observing elapsed time.

Conclusion

This project provided practical experience with developing and managing Linux kernel modules. The three modules demonstrated how to register initialization and cleanup functions, compile kernel code using Kbuild, load and remove modules, and inspect kernel messages.

The project also demonstrated how a module can expose information to user space through the `/proc` filesystem. The `proc-hello` module returned a fixed message, `proc-jiffies` exposed the current timer-tick counter, and `proc-seconds` used the difference between two counter values to calculate elapsed time.

Overall, the project reinforced the importance of correct resource management and careful testing when working with kernel code, which executes within the operating system's privileged environment.

References

1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, 10th Edition, Chapter 2, Programming Project: Introduction to Linux Kernel Modules.
2. Linux Kernel Documentation, The Linux Kernel Documentation.